

Functional Dependency: Cohesion and Coupling Practice

Section 1: Basic Functional Dependency

We are diving deep into two of the most critical concepts in software design: **Cohesion** and **Coupling**.

- **Cohesion** refers to the degree to which the elements inside a module belong together. Our goal is **High Cohesion**.
- **Coupling** refers to the degree of interdependence between software modules. Our goal is **Low Coupling**.

To help you master these concepts, I have designed two distinct practices for different groups in the lab.

Practice 1: The Design Auditor

Objective: Analyze an existing design to identify specific levels of cohesion and coupling and propose improvements.

Step-by-Step Instructions:

1. **Examine the Architecture:** Look at the provided Mermaid diagram of the "Library Management Module."
2. **Identify Cohesion:** Look at the `LibraryManager` class. Are the methods related to a single, well-defined task?
3. **Identify Coupling:** Look at the relationship between `LibraryManager` and `DatabaseConnection`. How much does one know about the other?
4. **Categorize:** Label the cohesion type (e.g., Coincidental, Functional) and coupling type (e.g., Content, Data).
5. **Propose a Solution:** Redraw the design to achieve higher cohesion and lower coupling.

Example Scenario: The "God Object" Library Manager

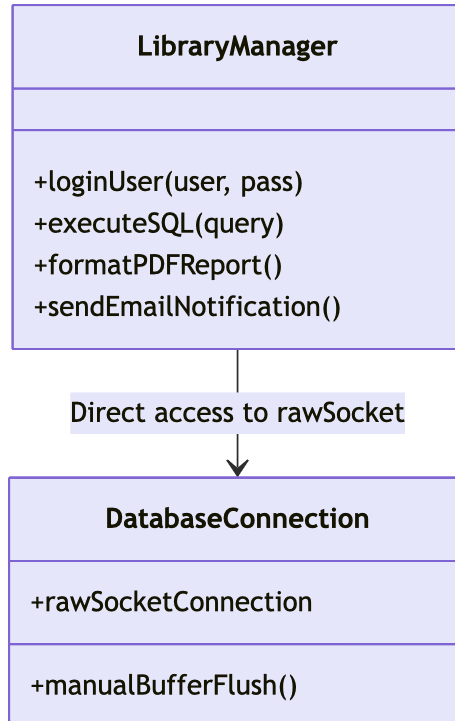
In this design, the `LibraryManager` handles user authentication, SQL queries, and printing reports all in one place.

Code snippet

classDiagram

```
class LibraryManager {  
    +loginUser(user, pass)  
    +executeSQL(query)  
    +formatPDFReport()  
    +sendEmailNotification()  
}  
class DatabaseConnection {  
    +rawSocketConnection  
    +manualBufferFlush()  
}
```

LibraryManager --> DatabaseConnection : Direct access to rawSocket



Solution for Practice 1:

- **Current State:** * **Cohesion:** *Coincidental/Logical Cohesion* (The methods are unrelated except that they are all "library things").
 - **Coupling:** *Content Coupling* (LibraryManager accesses the internal raw socket of the database connection).
- **The Audit Result:** This is a poor design. If the database buffer logic changes, the LibraryManager breaks. If the email protocol changes, the LibraryManager breaks.
- **Improved Design:** Split into AuthService, ReportGenerator, and a DatabaseWrapper. Use *Data Coupling* where only necessary parameters are passed.

Solution Description

In the initial unrefined system, the LibraryManager class suffered from **Coincidental Cohesion** because it combined completely unrelated duties (authentication, database queries, presentation formatting, and email network transport). Furthermore, it exhibited **Content Coupling** because it directly modified the internal, low-level properties (rawSocketConnection) of the DatabaseConnection class.

To resolve these issues, the system is refactored into distinct, single-responsibility components:

- **High Cohesion:** Splitting the "God Object" into isolated classes (AuthService, ReportGenerator, and NotificationService) ensures each class has exactly one reason to change.
- **Loose Coupling:** The DatabaseConnection class completely encapsulates its internal socket and buffer mechanisms. Other services interact with it exclusively through a clean, abstract public method (executeQuery()), achieving healthy **Data Coupling**.

Solution Diagram

Code snippet

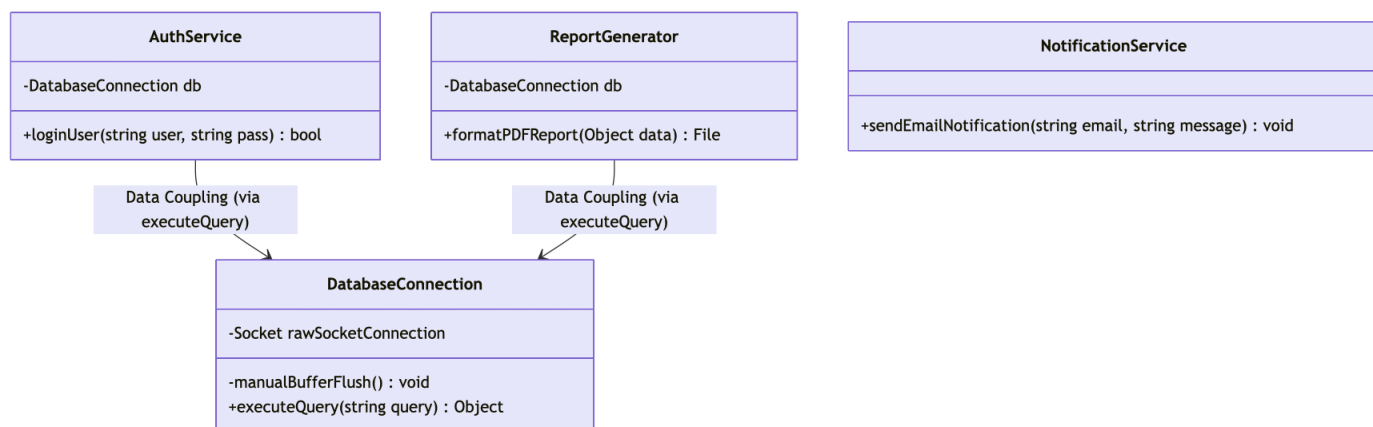
```
classDiagram
class DatabaseConnection {
    -Socket rawSocketConnection
    -manualBufferFlush() void
    +executeQuery(string query) Object
}

class AuthService {
    -DatabaseConnection db
    +loginUser(string user, string pass) bool
}

class ReportGenerator {
    -DatabaseConnection db
    +formatPDFReport(Object data) File
}

class NotificationService {
    +sendEmailNotification(string email, string message) void
}

AuthService --> DatabaseConnection : Data Coupling (via executeQuery)
ReportGenerator --> DatabaseConnection : Data Coupling (via executeQuery)
```



high cohesion : multiple objects, all do their own job

Low cohesion : God object

data coupling : limit access to each object to prevent spaghetti diagram

Loose coupling : independent object, doesn't have that much connection to each other

High/tight coupling : each object has too much connection with each other

Practice 2: The Refactoring Architect

Objective: Transform a tightly coupled, low-cohesion "Spaghetti" system into a modular, professional architecture.

Step-by-Step Instructions:

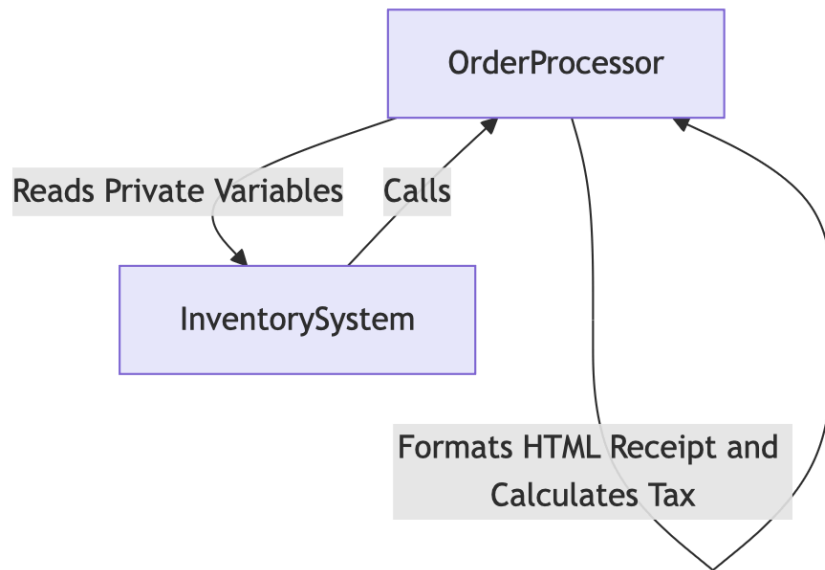
1. **Trace the Logic:** Follow the flow of the `OrderProcessor`.
2. **Isolate Responsibilities:** Identify "Concerns." (e.g., Validation, Calculation, Storage).
3. **Define Interfaces:** Create boundaries between these concerns so they communicate via simple data structures rather than internal state.
4. **Implement High Cohesion:** Group all "Calculation" logic into one module and all "Validation" logic into another.
5. **Implement Low Coupling:** Ensure the `OrderProcessor` doesn't know *how* the database saves data, only *that* it saves data.

Example Scenario: Tightly Coupled Order System

The `OrderProcessor` cannot function without the `InventorySystem` being initialized exactly a certain way.

Code snippet

```
graph TD
  A[OrderProcessor] -->|Reads Private Variables| B[InventorySystem]
  A -->|Formats HTML Receipt and Calculates Tax| A
  B -->|Calls| A
```



Solution for Practice 2:

- **Step 1:** Extract the `TaxCalculator` and `ReceiptGenerator` into their own classes (**Increase Cohesion**).
- **Step 2:** Use **Dependency Injection**. Pass the `InventorySystem` as an interface to the `OrderProcessor` (**Decrease Coupling**).

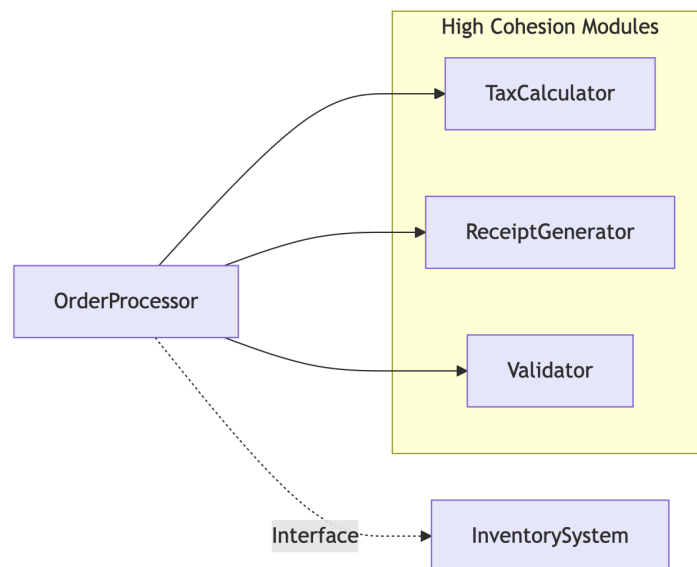
Improved Mermaid Architecture:

Code snippet

```

graph LR
    subgraph "High Cohesion Modules"
        TC[TaxCalculator]
        RG[ReceiptGenerator]
        VAL[Validator]
    end

    OP[OrderProcessor] --> VAL
    OP --> TC
    OP --> RG
    OP -. "Interface" .-> IS[InventorySystem]
  
```



- **The Refactoring Result:** * **Cohesion:** Now **Functional Cohesion**. Each module does exactly one thing.
 - **Coupling:** Now **Data/Stamp Coupling**. The modules share only the data they need to perform their specific tasks.

Summary for the Class

- **Practice 1** focused on **Discovery**: Learning to see the flaws in "spaghetti code."
- **Practice 2** focused on **Transformation**: Learning the structural moves required to fix those flaws.

Remember, class: A good engineer writes code that works. A **great** engineer writes code that can be changed without breaking the entire world. Happy coding!

Section 2: The "Refactoring" Squad

Goal: Take "spaghetti" logic and organize it into a clean, modular system.

We are moving beyond just "making code work." We are learning how to make code **maintainable**. In software engineering, we strive for two things:

1. **High Cohesion:** A module should do **one thing** and do it well.
2. **Low Coupling:** Modules should be as **independent** as possible.

Below are two distinct practice sets. Each starts at a beginner level and builds toward an advanced architectural challenge.

Practice 3: Identifying the "God Object" (Beginner)

The Scenario: You are given a `SmartHomeManager` class. It controls the lights, handles user billing, and generates PDF weather reports.

Step-by-Step:

1. **Analyze:** Identify why this class has low cohesion.
2. **Decompose:** Identify the three distinct responsibilities.
3. **Refactor:** Create three separate classes with specific names.

Problem Diagram:

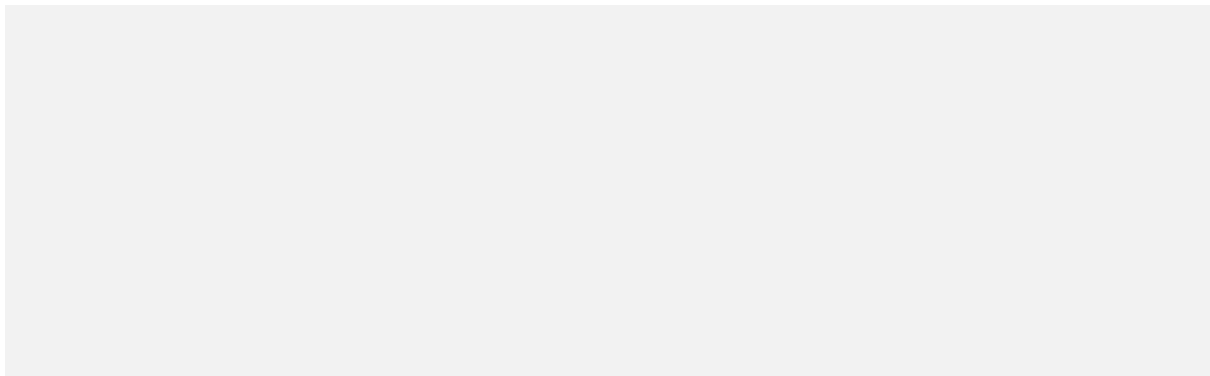
Code snippet

```
classDiagram
class SmartHomeManager {
    +turnOnLights()
    +processSubscriptionPayment()
    +getWeatherForecast()
    +generatePDFBill()
}
```

SmartHomeManager
+turnOnLights() +processSubscriptionPayment() +getWeatherForecast() +generatePDFBill()

Solution: The original class has **Coincidental Cohesion**—the functions are unrelated. We refactor into **Functional Cohesion**.

Code snippet



Practice 4: Breaking Content Coupling (Advanced)

The Scenario: The LightController directly modifies a private variable inside the ElectricityGrid class. This is Content Coupling (the worst kind!).

Step-by-Step:

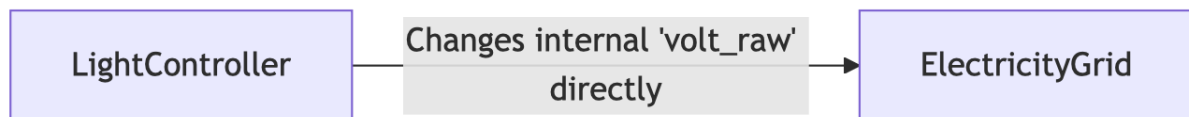
1. **Identify:** Find where one class "reaches into" the internals of another.
2. **Abstract:** Create a method (Setter or Interface) so the LightController doesn't need to know the internal variable names.

Problem Diagram:

Code snippet

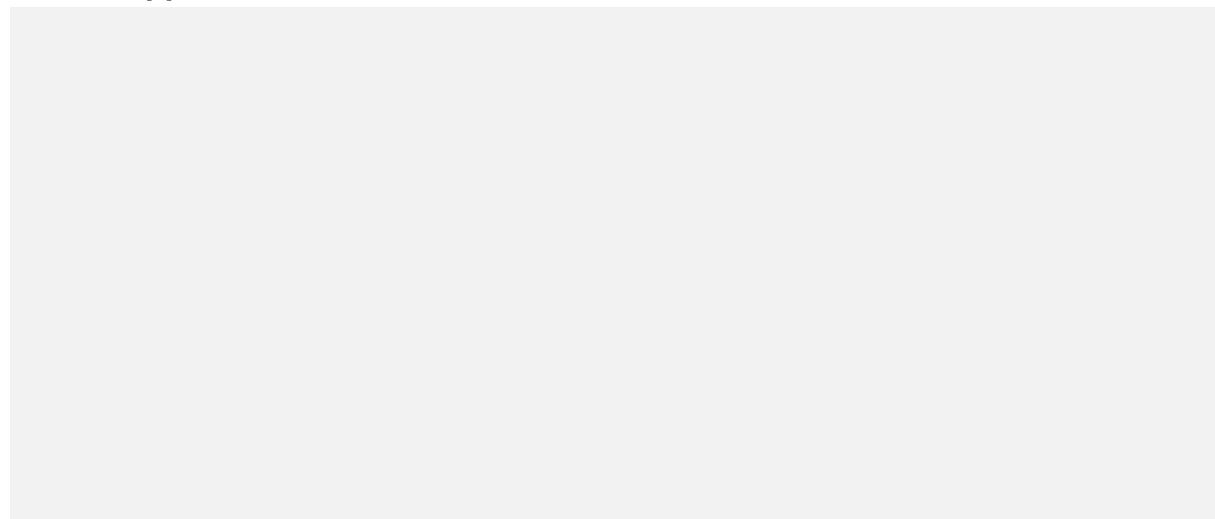
graph LR

```
LC[LightController] -- "Changes internal 'volt_raw' directly" --> EG[ElectricityGrid]
```



Solution: We move to **Data Coupling** by using a public method.

Code snippet



Section 3: The "Architecture" Squad

Goal: Design communication patterns that minimize dependencies.

Practice 5: From Stamp to Data Coupling (Beginner)

The Scenario: You have a `Payroll` system. To calculate a tax deduction, you pass the entire `Employee` object (containing their address, bio, and photo) to a `TaxCalculator` that only needs their salary.

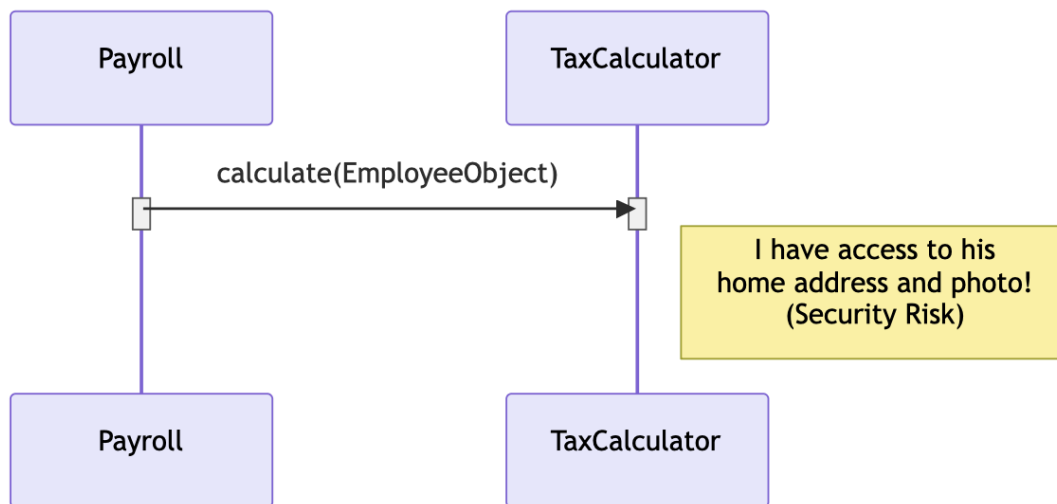
Step-by-Step:

1. **Evaluate:** Determine why passing the whole object is "Stamp Coupling."
2. **Minimize:** Change the function signature to accept only a float salary.

Problem Diagram:

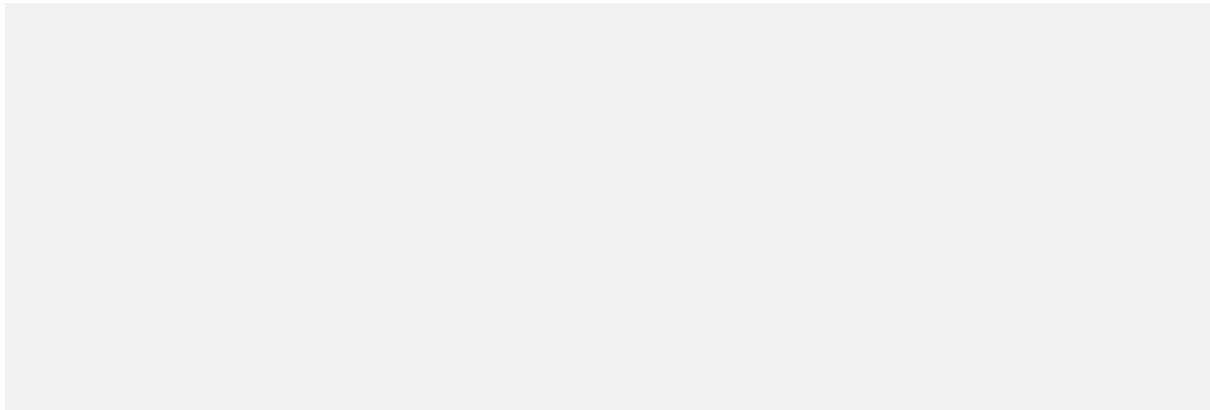
Code snippet

```
sequenceDiagram
    Payroll->>TaxCalculator: calculate(EmployeeObject)
    activate Payroll
    activate TaxCalculator
    deactivate Payroll
    deactivate TaxCalculator
    Note right of TaxCalculator: I have access to his <br/>home address and photo! <br/>(Security Risk)
```



Solution: Reduce the interface to the bare minimum.

Code snippet



Practice 6: Implementing Observer Pattern (Advanced)

The Scenario: A `StockMarket` class needs to update a `MobileApp`, a `WebDashboard`, and a `Database`. Currently, the `StockMarket` class has hardcoded references to all three.

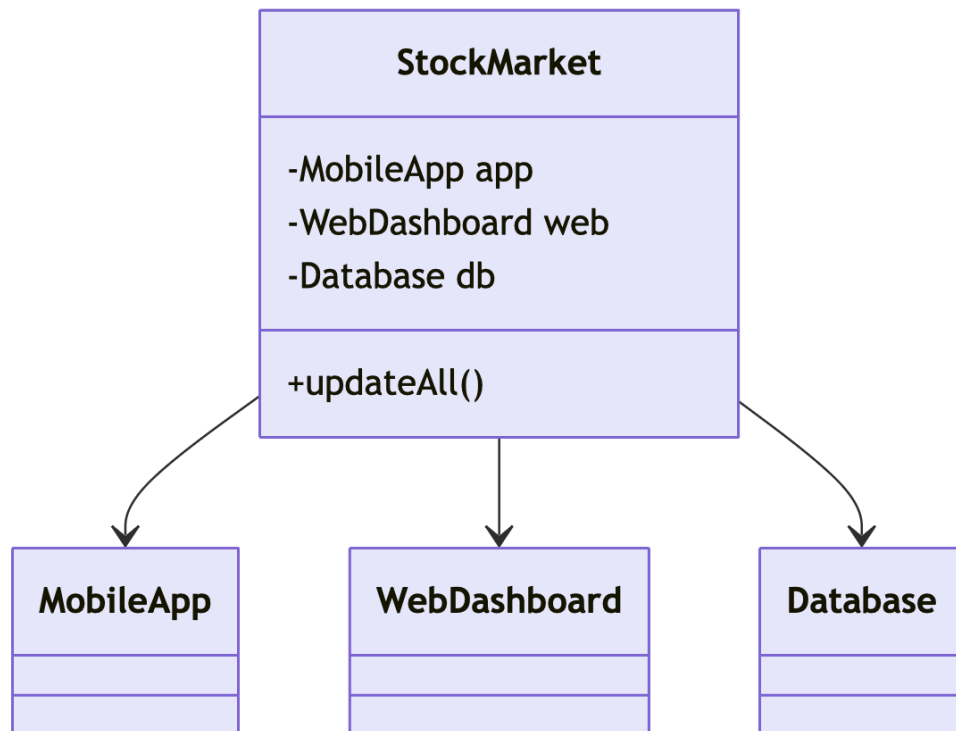
Step-by-Step:

1. **Recognize:** The `StockMarket` is "tightly coupled" to three different platforms.
2. **Invert:** Use an **Interface (Observer Pattern)** so the `StockMarket` doesn't know *who* is listening.

Problem Diagram:

Code snippet

```
classDiagram
class StockMarket {
  -MobileApp app
  -WebDashboard web
  -Database db
  +updateAll()
}
StockMarket --> MobileApp
StockMarket --> WebDashboard
StockMarket --> Database
```



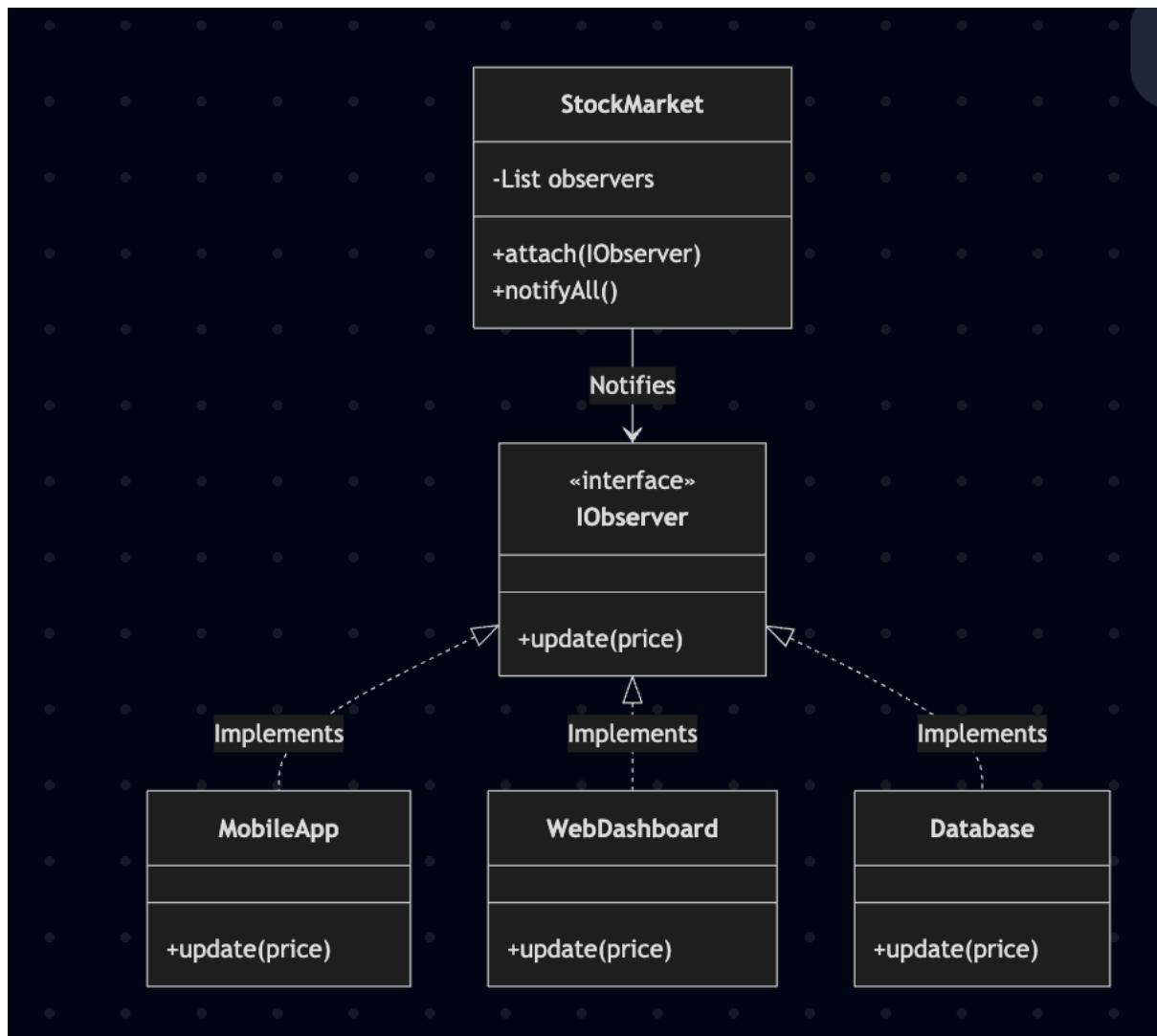
Solution: By using an `Observer` interface, we achieve **Low Coupling**.
The `StockMarket` only knows about the interface, not the specific classes.

Code snippet

```
classDiagram
class IObservable {
    <<interface>>
    +update(price)
}
class StockMarket {
    -List<IObservable> observers
    %%IObservable is there just to type cast
    +attach(IObservable)
    +notifyAll()
}
class MobileApp {
    +update(price)
}
class WebDashboard {
    +update(price)
}
class Database {
    +update(price)
}

StockMarket --> IObservable : Notifies
```

```
IObserver <|.. MobileApp : Implements  
IObserver <|.. WebDashboard : Implements  
IObserver <|.. Database : Implements
```



Section 3: Advanced Functional Dependency

Let's build upon our foundation of modular design. To ensure both groups get thorough exposure to both sides of the coin, I have prepared two additional practices.

- **Practice 7** focuses entirely on moving from **Low Cohesion** to **High Cohesion** within data-processing logic.
- **Practice 8** focuses entirely on refactoring **Tight Coupling (Control & Concrete Dependencies)** into **Loose Coupling** using interfaces.

Practice 7: Refactoring the "All-in-One" Data Processor

Objective: Identify **Logical & Sequential Cohesion** flaws in a data workflow and refactor them into highly cohesive, single-responsibility units.

The Scenario

You are building an automated system to process monthly electricity metrics for a smart campus. A developer handed you a class named `ElectricityBillProcessor`. It reads raw usage metrics from a JSON file, executes localized tariff calculations, formats a customer-facing HTML report, and saves it to a deployment bucket.

While the code runs fine sequentially, it has **low cohesion** because it mixes file I/O, business logic calculations, and presentation rendering inside a single class.

Step-by-Step Instructions for Students

1. **Isolate the Domains:** Draw lines between Data Access (I/O), Domain Logic (Math/Rules), and Presentation (UI/Formatting).

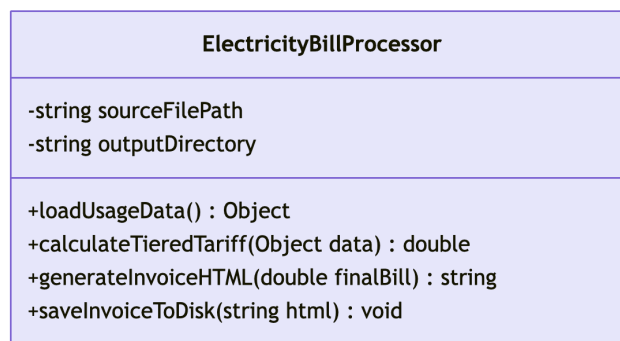
2. **Separate Responsibilities:** Break the single large class into three distinct, highly focused classes.
3. **Define Immutable Data Flow:** Ensure data flows cleanly from one class to the next without any class crossing into another's domain.

Problem Architecture (Low Cohesion)

Code snippet

classDiagram

```
class ElectricityBillProcessor {  
    -string sourceFilePath  
    -string outputDirectory  
    +loadUsageData() Object  
    +calculateTieredTariff(Object data) double  
    +generateInvoiceHTML(double finalBill) string  
    +saveInvoiceToDisk(string html) void  
}
```



Solution for Practice 7

The original class exhibits **Sequential Cohesion** (the output of one part is the input to the next), but overall low functional cohesion. If the database schema, tax law, or UI layout changes, this single class must be rewritten.

We refactor this into three highly cohesive classes where each has exactly **one reason to change**.

Code snippet

classDiagram

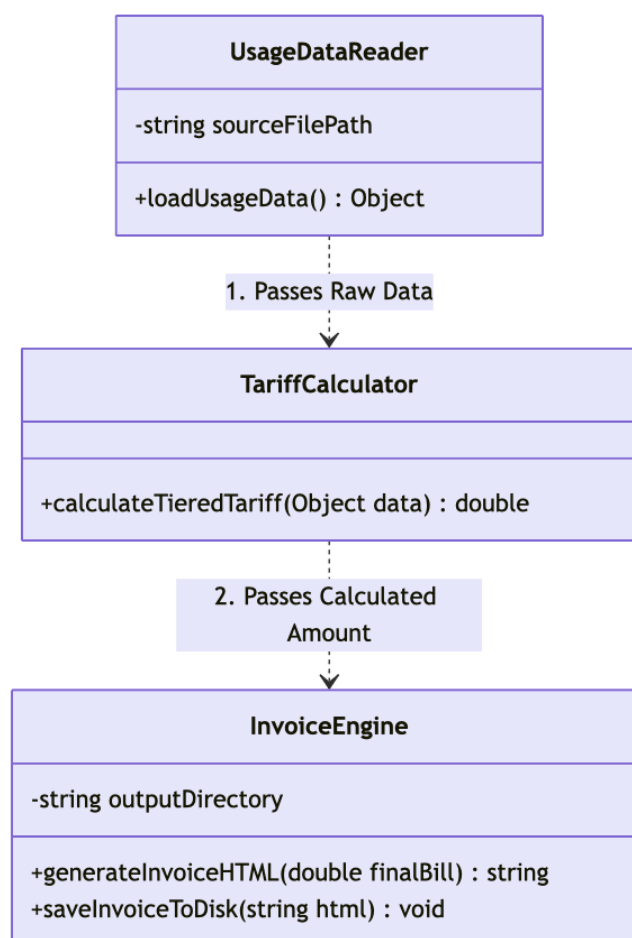
```
class UsageDataReader {  
    -string sourceFilePath  
    +loadUsageData() Object  
}
```

```
class TariffCalculator {
    +calculateTieredTariff(Object data) double
}
```

```
class InvoiceEngine {
    -string outputDirectory
    +generateInvoiceHTML(double finalBill) string
    +saveInvoiceToDisk(string html) void
}
```

UsageDataReader ..> TariffCalculator : 1. Passes Raw Data

TariffCalculator ..> InvoiceEngine : 2. Passes Calculated Amount



- **Result: High Functional Cohesion.** TariffCalculator handles pure math and can be unit-tested seamlessly without mocking file systems or HTML rendering.

Practice 8: Eliminating Control Coupling in Notification Routing

Objective: Transform a tightly coupled switch-case management structure (**Control Coupling**) into a polymorphically open, loosely coupled layout.

The Scenario

Your DevOps deployment pipeline needs to send alerts whenever a build fails. A developer created a `NotificationService` that handles alerts across various infrastructure channels (Email, SMS, and Slack).

To select a channel, the client application passes a strict control flag parameter (1 for Email, 2 for SMS, 3 for Slack). The `NotificationService` then uses a switch-case statement and instantiates hardcoded third-party API clients directly inside the logic.

This is **Tightly Coupled** because:

- **Control Coupling:** The caller dictates the internal execution flow of the method using a magic flag.
- **Concrete Dependency:** The engine cannot compile or function without direct access to specific vendor clients (`TwilioSMS`, `SendGridEmail`).

Step-by-Step Instructions for Students

1. **Identify the Control Flag:** Look for conditional statements (`if/else` or `switch`) that process execution modes.
2. **Introduce an Abstraction:** Create a common interface layer that defines a unified action contract.
3. **Apply Open-Closed Principle:** Redesign the engine so new notification vendors can be plugged in without editing the core routing logic.

Problem Architecture (Tight Coupling)

Code snippet

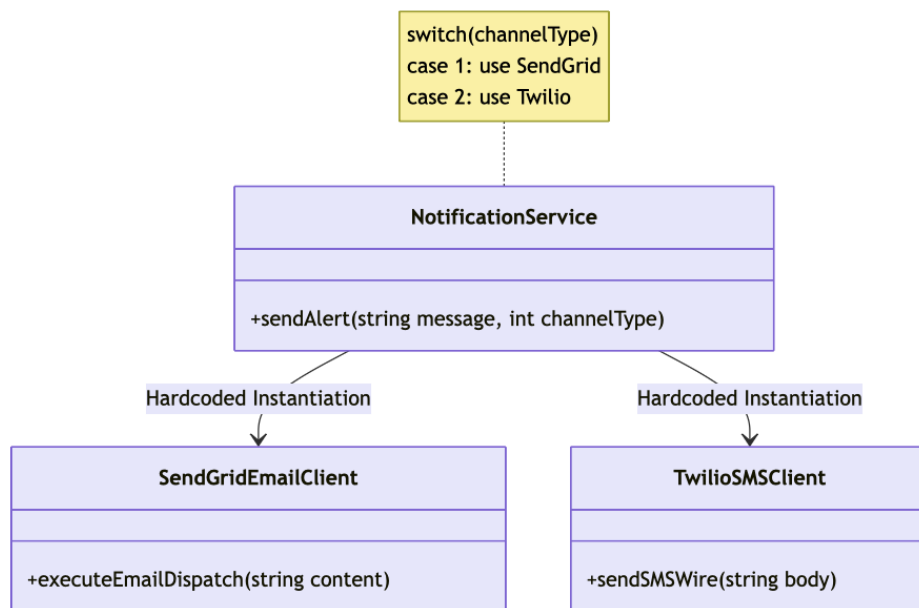
```
classDiagram
```

```

class NotificationService {
    +sendAlert(string message, int channelType)
}
class SendGridEmailClient {
    +executeEmailDispatch(string content)
}
class TwilioSMSClient {
    +sendSMSWire(string body)
}

```

NotificationService --> SendGridEmailClient : Hardcoded Instantiation
 NotificationService --> TwilioSMSClient : Hardcoded Instantiation
 note for NotificationService "switch(channelType)
 case 1: use SendGrid
 case 2: use Twilio"



Solution for Practice 4

We resolve the tight coupling by introducing a **Strategy Pattern** via an interface (**NotificationProvider**). Instead of passing an execution control flag, we inject the appropriate provider.

Code snippet

```

classDiagram
class NotificationService {
    -List~NotificationProvider~ providers
    +registerProvider(NotificationProvider p)
    +broadcast(string message)
}

```

```

}

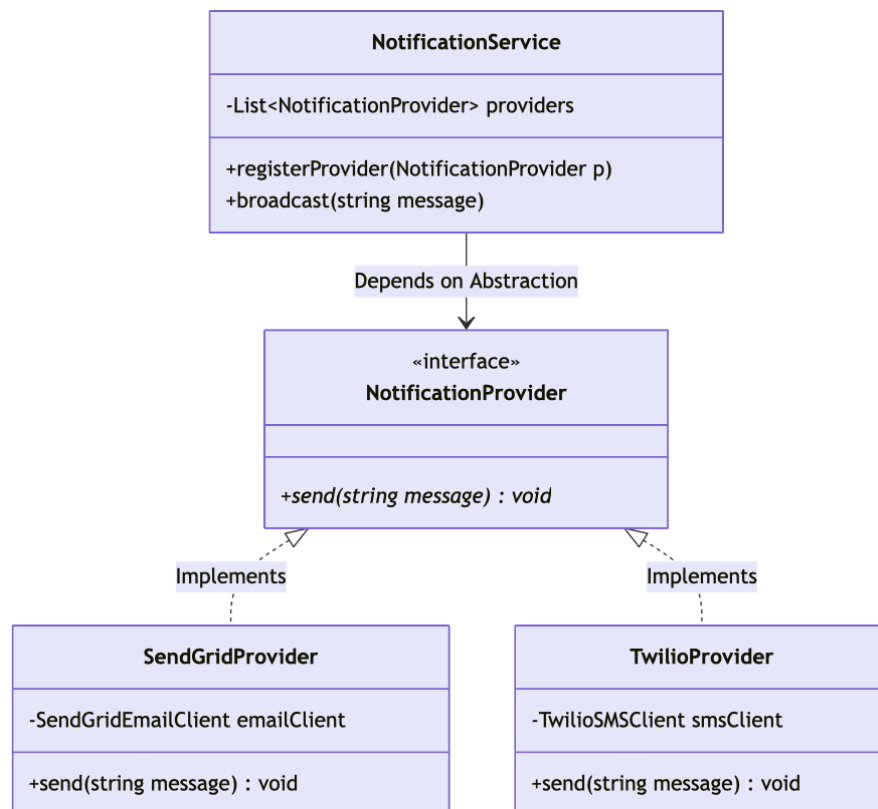
class NotificationProvider {
  <<interface>>
  +send(string message)* void
}

class SendGridProvider {
  -SendGridEmailClient emailClient
  +send(string message) void
}

class TwilioProvider {
  -TwilioSMSClient smsClient
  +send(string message) void
}

```

NotificationService --> NotificationProvider : Depends on Abstraction
 NotificationProvider <|.. SendGridProvider : Implements
 NotificationProvider <|.. TwilioProvider : Implements



- **Result: Loose Data/Interface Coupling.** The `NotificationService` no longer cares *how* a notification is sent or *who* sends it. It only knows that any registered provider conforms to the `.send()` contract.